1. Identify Program Components

- **Data Structure:** Array-based Stack operating on the LIFO (Last-In, First-Out) principle.
- **Variables:**
  - stack: An integer array of size 5 representing the stack storage.
  - top: An integer representing the index of the top element.
  - item: An integer in main() to store popped values and push arguments.
  - i: A loop counter variable inside display().
- **Initial Values:** SIZE = 5, top = -1.
- **Functions:**
  - push(int item): Adds an item to the stack.
  - pop(): Removes and returns the top item.
  - display(): Iterates through and prints current stack elements.
- **Conditions:** if (top == SIZE - 1) for Overflow check; if (top == -1) for Underflow/Empty check.
- **Loops:** for (i = 0; i <= top; i++) to iterate from the bottom to the top of the stack during display.

2 & 3. Step-by-Step Trace Table & Function Calls

Here is the manual execution trace tracking variables, states, and outputs step-by-step:

Step	Operation / Call	top (After)	Logical Stack State	Output Printed	Notes (i values)
Initial	Setup	-1	[ ]	*(none)*	SIZE = 5
1	push(10)	0	[10]	Pushed: 10	
2	push(20)	1	[10, 20]	Pushed: 20	
3	push(30)	2	[10, 20, 30]	Pushed: 30	
4	display()	2	[10, 20, 30]	Stack: 10 20 30	i = 0, 1, 2
5	item = pop()	1	[10, 20]	Popped: 30	Returned 30
6	push(40)	2	[10, 20, 40]	Pushed: 40	Overwrites old 30

7	push(50)	3	[10, 20, 40, 50]	Pushed: 50	
8	push(60)	4	[10, 20, 40, 50, 60]	Pushed: 60	top reaches SIZE-1
9	push(70)	4	[10, 20, 40, 50, 60]	Stack Overflow	Fails overflow check
10	display()	4	[10, 20, 40, 50, 60]	Stack: 10 20 40 50 60	i = 0, 1, 2, 3, 4
11	item = pop()	3	[10, 20, 40, 50]	Popped: 60	Returned 60
12	item = pop()	2	[10, 20, 40]	Popped: 50	Returned 50
13	item = pop()	1	[10, 20]	Popped: 40	Returned 40
14	item = pop()	0	[10]	Popped: 20	Returned 20
15	item = pop()	-1	[ ]	Popped: 10	Returned 10
16	display()	-1	[ ]	Stack is Empty	top == -1 triggers
17	item = pop()	-1	[ ]	Stack Underflow Popped: -1	pop returns -1.

4. Predicted Output and Final State

Predicted Console Output:

Pushed: 10
Pushed: 20
Pushed: 30
Stack: 10 20 30
Popped: 30
Pushed: 40
Pushed: 50
Pushed: 60
Stack Overflow
Stack: 10 20 40 50 60
Popped: 60
Popped: 50
Popped: 40
Popped: 20
Popped: 10
Stack is Empty
Stack Underflow
Popped: -1

Final Stack State: top = -1. The stack is logically empty (though the physical array memory still contains the residual integers [10, 20, 40, 50, 60]).

5. Overflow and Underflow Triggers

- **Stack Overflow:** Caused by push(70); when top is already 4 (SIZE - 1). The stack capacity of 5 is exceeded.
- **Stack Underflow:** Caused by the final item = pop(); call at the end of main(). All elements have been removed, making top = -1.

6. Modified Trace

Modification: Replace push(70); with an extra pop();.

- **Original Step 9:** push(70) triggers "Stack Overflow". Stack remains size 5.
- **Modified Step 9:** pop() executes, returning 60. top drops to 3.
- **New Output Ripple Effect:** The next display() would print Stack: 10 20 40 50. The final sequence of pops will empty the stack faster, causing two underflows at the very end of the program instead of one.

7. Logical Errors and Limitations

- **Limitation - Fixed Size:** The array size is hardcoded to 5. It cannot dynamically scale up to accommodate more data.
- **Logical Flaw - Return Value Ambiguity:** When underflow occurs, pop() returns -1. If a user intentionally pushes -1 onto the stack and then pops it, the calling function cannot

distinguish between a successful pop of -1 and an underflow error.

- *Correction:* Pass the popped item back via a pointer argument, and return a boolean/status code from the pop() function (e.g., int pop(int *item)).

8. Execution Validation

(Simulated execution validates the trace). The predicted output matches exactly with the deterministic behavior of the provided C program.

9. Additional Test Cases

Test Case Intent	Actions	Expected Output
1. Immediate Underflow	Empty main(), call pop()	Stack Underflow, returns -1
2. Rapid Fill	Loop push(1) 6 times	5 times Pushed: 1, 1 time Stack Overflow
3. Ambiguity Check	push(-1); item = pop();	Pushed: -1 Popped: -1 (No underflow printed)

10. Conclusion

This program effectively demonstrates a standard contiguous LIFO (Last-In, First-Out) stack implementation. Tracking the top index variable is the critical mechanism controlling logical operations, effectively shielding the user from "garbage" memory data without needing to explicitly erase deleted values. While mathematically sound, a real-world implementation would require dynamic memory allocation (like Linked Lists or malloc) and stricter error-code handling for production environments.